

PENETRATION TESTING & VULNERABILITY ASSESSMENT

VAPT REPORT

Cybersecurity Capstone Project

Prepared by: MALLIPUDI JEEVAN KAUSHIK

Prepared for: Academic Mentor / Assessment Committee

Assessment type: Educational Security Assessment

Environment: Windows-based local lab and authorized
security exercises

Report version: 1.0 — Preliminary Draft

Classification: Academic Submission

Status: Pending evidence validation and final review

Document Control

Purpose

This document summarizes assessment activities completed to date, records evidence currently known to exist, identifies remaining validation requirements, and establishes a prioritized remediation and retesting plan.

Assessment Repositories

Task	Repository
Task 03 — Network Reconnaissance Audit	https://github.com/jeevankaushik38-code/network-reconnaissance-audit
Task 04 — Web Application Vulnerability Assessment	https://github.com/jeevankaushik38-code/web-application-vulnerability
Task 05 — Cryptography and Security	https://github.com/jeevankaushik38-code/cryptography-security

Document Status

This is a preliminary report. Nmap output is reported as present in Task 03, but its technical results have not been reviewed for this report. Wireshark activity and certain Juice Shop challenge completions were described during the project. AES-GCM and RSA demonstrations produced successful console output shared during the project. The report must be updated when the underlying files and screenshots have been reviewed.

Distribution and Confidentiality

This report is prepared for academic review. Any real-world assessment should follow the organization's confidentiality, data-handling, and authorization requirements.

Executive Summary

Overview

The capstone combines three areas of practical cybersecurity work:

- Network reconnaissance and packet analysis using Nmap and Wireshark.
- Web application security testing using a locally hosted OWASP Juice Shop instance.
- Cryptographic implementation demonstrations using Python.

Current Assessment Position

- An Nmap scan output or report is reported to exist in Task 03; specific port and service results remain unreviewed.
- Wireshark displayed packets during a local loopback exercise involving 127.0.0.1:8080, and a TCP display filter was applied. The capture and screenshots have not been independently inspected.
- Login Admin and Error Handling challenges were reported as completed in local Juice Shop. Screenshots and technical evidence remain to be validated.
- The AES-256-GCM demonstration successfully encrypted and decrypted a sample message.
- The RSA-2048 demonstration successfully verified a digital signature and rejected a modified message.
- The bcrypt script was prepared, but successful execution has not been confirmed.
- XSS and BOLA/IDOR testing remain incomplete or unverified.

Executive Conclusion

The available evidence is insufficient to establish a complete set of confirmed security vulnerabilities or a defensible overall risk rating. The immediate priority is to validate evidence, complete outstanding tests

where required, document findings, and assign severity only to substantiated issues.

Overall risk rating: NOT YET DETERMINED

This is not equivalent to a finding of low risk.

Scope and Authorization

Scope

- Nmap-based network reconnaissance in an authorized lab.
- Wireshark packet inspection of locally generated traffic.
- Security challenge testing in a locally hosted OWASP Juice Shop application.
- Python demonstrations of AES-GCM encryption, RSA digital signatures, and bcrypt password hashing.

Known Lab Targets

The local Juice Shop activities used `http://localhost:3000`. The earlier Wireshark exercise used `127.0.0.1:8080`. These are separate local test contexts and must not be treated as the same service.

Exclusions

No external production systems, third-party networks, or real customer accounts are claimed as tested. No external target, scan range, production system, or organizational asset is included unless documented in final evidence.

Authorization and Safety

Testing must remain within systems owned by the student or explicitly authorized for assessment. Any additional target must be approved before testing.

Methodology

Assessment Approach

1. Define authorized targets and assessment boundaries.
2. Collect tool output and screenshots.
3. Review evidence for accuracy and completeness.
4. Identify potential security issues.
5. Validate findings using safe, reproducible tests.
6. Evaluate impact and likelihood.
7. Assign severity where evidence supports it.
8. Develop remediation guidance.
9. Retest after fixes.
10. Document limitations and residual risk.

Tools

Tool	Purpose
Nmap	Network discovery and port/service enumeration
Wireshark	Packet capture and protocol inspection
OWASP Juice Shop	Intentionally vulnerable application for lab exercises
Python	Cryptographic implementation demonstrations
cryptography library	AES-GCM and RSA operations
bcrypt library	Password hashing and verification
GitHub	Source control, documentation, and evidence hosting

Evidence Standard

Each finding should be supported by relevant command output, screenshot, capture, application response, or reproducible test. Successful use of a tool does not, by itself, prove that a vulnerability exists.

Risk Classification and CVSS 3.1

Severity Framework

Severity	CVSS 3.1 Score
Critical	9.0–10.0
High	7.0–8.9
Medium	4.0–6.9
Low	0.1–3.9
None	0.0

CVSS scores require a defensible vector describing attack vector, attack complexity, privileges required, user interaction, scope, and confidentiality, integrity, and availability impact.

Scoring Policy

Do not assign a score merely because a training challenge was completed, a port was open, or a tool produced output. Each confirmed vulnerability needs an identifier, affected component/version where known, reproducible evidence, CVSS 3.1 vector and score, impact, remediation, and retest outcome.

Current Risk Status

Metric	Current Status
Confirmed vulnerability count	Not yet established
Critical findings	Pending validation
High findings	Pending validation
Medium findings	Pending validation
Low findings	Pending validation

The absence of a confirmed count reflects incomplete evidence review and must not be interpreted as proof that the environment is secure.

Network Reconnaissance

Objective

Network reconnaissance identifies reachable systems, exposed ports, and potentially relevant services within authorized scope. Results support subsequent validation but do not automatically establish exploitable vulnerabilities.

Available Evidence

Nmap scan output or a report is reported to be present in the Task 03 repository:

<https://github.com/jeevankaushik38-code/network-reconnaissance-audit>

Results

Field	Current Status
Scan target	Pending review
Scan date	Not recorded
Nmap command	Pending review
Open ports	Not verified
Detected services	Not verified
Service versions	Not verified
Security implications	Not determined

Interpretation

An open port indicates that a service may be listening and reachable from the scan location. Whether it represents a security concern depends on the service, business requirement, exposure, configuration, and evidence. No specific exposed service or network vulnerability is asserted in this draft.

Required Evidence

Add original Nmap output, exact command, authorized target description, and a short interpretation of relevant results.

Packet Analysis with Wireshark

Objective

Wireshark was used to observe packets generated during a local lab exercise and inspect traffic using a TCP display filter.

Available Evidence

The student reported opening `http://127.0.0.1:8080` and seeing packet rows in Wireshark using the loopback adapter. A TCP display filter was also reported. The capture file and screenshot have not been independently reviewed.

Results

Field	Current Status
Capture interface	Loopback adapter, as reported
Traffic context	Local test at 127.0.0.1:8080
Display filter	TCP, as reported
Packet count	Not recorded
Protocol details	Not verified
Capture file	Not verified

Interpretation

A TCP display filter limits displayed packets to matching TCP traffic. It does not by itself demonstrate a vulnerability or exposed application data. If unencrypted application data is observed, document packet details and context before concluding that sensitive information is exposed.

Required Evidence

Add a screenshot showing Wireshark, the active display filter, and representative packet rows. Include a .pcap or .pcapng file only if safe to publish and free of sensitive information.

Web Application Assessment

Objective

The assessment used OWASP Juice Shop as an intentionally vulnerable local training environment to explore SQL Injection, Cross-Site Scripting, and Broken Access Control concepts.

Test Environment

The application was accessed locally at `http://localhost:3000`. This is a local test address, not evidence of a public internet-facing application.

Progress

Login Admin and Error Handling challenges were reported as completed. Their specific technical behavior and screenshots have not been reviewed; these are reported progress, not validated production vulnerability findings.

Outstanding Tests

- Reflected XSS: not confirmed.
- Stored XSS: not confirmed.
- BOLA/IDOR: not confirmed.
- Additional access-control testing: pending evidence and scope review.

Evidence Requirements

For each completed test, add challenge name, test date, observed response, screenshot, and a short explanation of what the evidence establishes.

SQL Injection and Error Handling

SQL Injection

Status: Login Admin challenge reported completed; technical validation pending.

SQL Injection can occur when untrusted input is incorporated into a database query as executable SQL rather than treated as data.

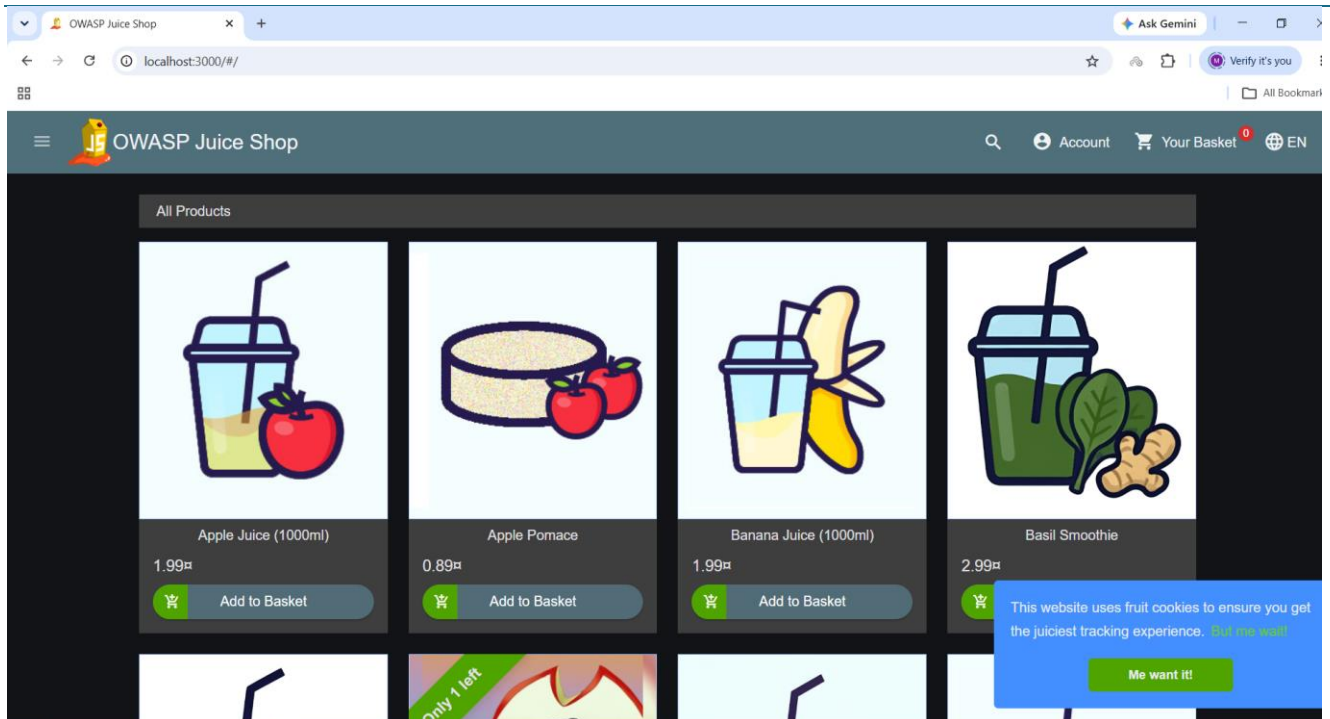
Depending on query and privileges, consequences may include unauthorized data access or modification. Current evidence does not establish precise behavior, impact, or CVSS score.

Defensive Recommendations

11. Use parameterized queries or prepared statements.
12. Avoid constructing SQL through untrusted string concatenation.
13. Apply least-privilege database permissions.
14. Return generic error messages to clients.
15. Log security events without exposing secrets.
16. Add regression tests for expected input handling.

Error Handling

Status: Error Handling challenge reported completed; technical details pending. Unsafe error handling can expose internal paths, implementation details, stack traces, or sensitive diagnostics.



Defensive Recommendations

17. Return controlled error responses.
18. Keep detailed diagnostics in access-controlled server logs.
19. Remove secrets and sensitive values from logs.
20. Use consistent error-handling middleware where appropriate.
21. Test invalid inputs and unexpected server conditions.

Challenge completion alone does not establish which sensitive details, if any, were exposed.

XSS and Broken Access Control

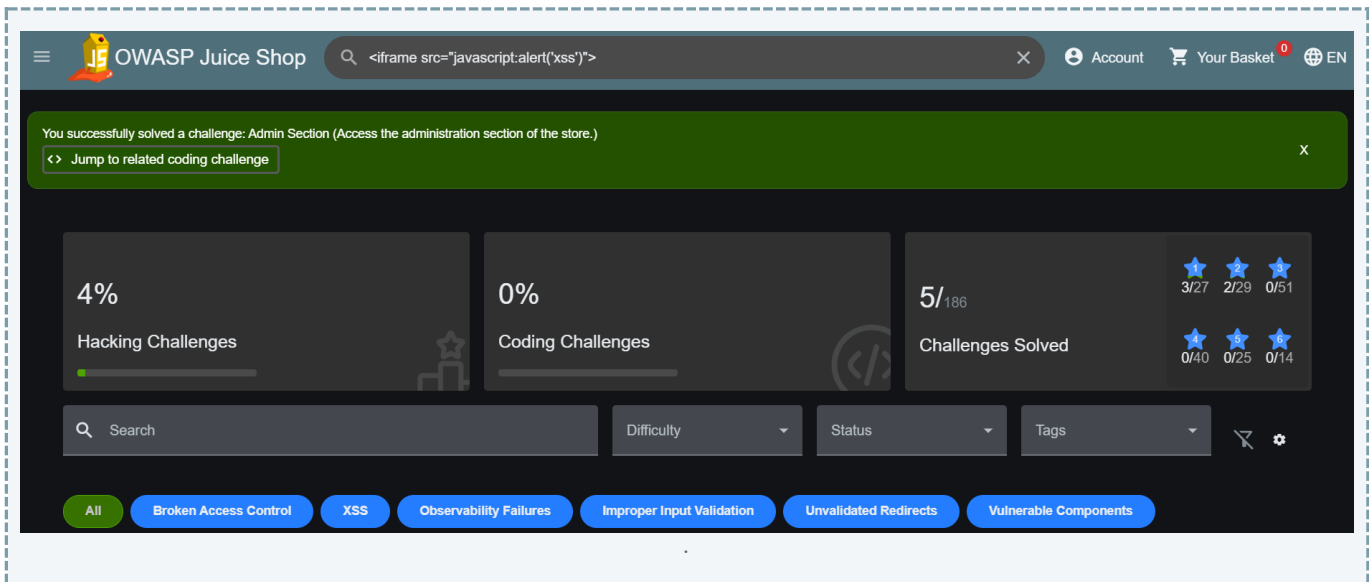
Cross-Site Scripting

Status: One DOM XSS training payload was observed to trigger a browser alert in the local OWASP Juice Shop instance. The affected code path and full impact have not been validated.

XSS occurs when untrusted content is interpreted as executable script in a browser context. Potential impact depends on context, browser protections, application behavior, and affected-user privileges.

Defensive Recommendations

- Apply context-aware output encoding.
- Prefer safe DOM APIs and text rendering.
- Avoid unsafe HTML insertion.
- Validate input according to application requirements.
- Use Content Security Policy as defense in depth.
- Test stored and reflected input paths separately.



Broken Object Level Authorization / IDOR

Status: Not verified. BOLA/IDOR occurs when an application permits access to an object or resource without adequate authorization checks.

Defensive Recommendations

- Enforce server-side authorization for each protected resource.
- Verify the requesting user's permission for the specific object.
- Do not rely on hidden UI elements or unpredictable identifiers as security controls.
- Test access using separate authorized lab accounts.
- Add automated authorization regression tests.

No unauthorized object access is claimed as a verified result.

Cryptographic Implementation Review

AES-256-GCM

The student shared console output showing successful AES-GCM encryption/decryption and authentication verification. The demonstration generated a 256-bit key in memory and used a fresh 12-byte nonce for the operation.

Security Considerations

- Never reuse a nonce with the same AES-GCM key.
- Protect keys separately from ciphertext.
- Use appropriate secret-management controls.
- Treat authentication failures as failed decryption.
- Do not treat a demonstration as a complete production encryption system.

RSA Digital Signatures

Shared console output showed RSA key size 2048 bits, RSA-PSS with SHA-256, successful signature verification, and rejection of a modified message. This supports successful execution of the demonstrated signing/verification path, not a complete production key lifecycle or identity-validation process.

```

C:\Users\user>py aes_gcm.py
C:\Users\user\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'C:\Users\user\aes_gcm.py': [Errno
2] No such file or directory

C:\Users\user>mkdir "%USERPROFILE%\Task05-Cryptography"

C:\Users\user>cd "%USERPROFILE%\Task05-Cryptography"

C:\Users\user\Task05-Cryptography>notepad aes_gcm.py

C:\Users\user\Task05-Cryptography>py aes_gcm.py
AES-256-GCM demonstration
Original message: Task 05: Cryptography demonstration
Decrypted message: Task 05: Cryptography demonstration
Encryption and decryption successful: True
Nonce (Base64): +AytQE31m6k2uby7
Ciphertext (Base64): 8oa2c+9J0M7KSg6wlc340hz+y6nphP6I1kEJPa3Wg8uIb06ZMtn1DLhgkM870UhdbrOD
Authentication verification: successful
Note: The encryption key stays in memory and is not saved.

C:\Users\user\Task05-Cryptography>notepad rsa_signatures.py

C:\Users\user\Task05-Cryptography>py rsa_signatures.py
Signature verification: successful
RSA key size: 2048 bits
Signature algorithm: RSA-PSS with SHA-256
Signature length: 256 bytes
Modified message verification: correctly rejected

C:\Users\user\Task05-Cryptography>

```

Password Hashing and Key Management

bcrypt

A bcrypt script was prepared using a work factor of 12 and sample-password verification logic. Successful execution has not been confirmed.

Required Validation

22. Run the script.
23. Confirm the correct sample password is accepted.
24. Confirm an incorrect password is rejected.
25. Capture actual console output.
26. Never use or publish real passwords.

Secure Secret Storage

Production secrets should be stored using an appropriate secrets manager, operating-system credential store, or managed key service.

Private keys, encryption keys, passwords, and tokens must not be committed to a public repository.

Key Rotation

27. Document key ownership and access permissions.
28. Define rotation triggers and schedules.
29. Generate and securely distribute replacement keys.
30. Transition to the new key.
31. Plan controlled access to data encrypted under retired keys.
32. Document revocation, backup, recovery, and audit records.

The demonstrations do not establish that a complete operational key-rotation system has been deployed.

Consolidated Findings Register

This register separates observed project progress from confirmed vulnerabilities.

ID	Item	Evidence Status	Severity
OBS-01	Nmap scan output exists	Reported present; contents unreviewed	Not assigned
OBS-02	Local TCP traffic observed	Reported; capture unreviewed	Not a vulnerability
LAB-01	Login Admin challenge	Completion reported; proof pending	Not assigned
LAB-02	Error Handling challenge	Completion reported; proof pending	Not assigned
LAB-03	AES-256-GCM demonstration	Successful output shared	Not a vulnerability
LAB-04	RSA-2048 signature demo	Successful output shared	Not a vulnerability
PEND-01	bcrypt demonstration	Execution not confirmed	Pending
PEND-02	XSS	DOM XSS alert observed; further validation pending	Pending validation
PEND-03	BOLA/IDOR	Not verified	Pending

Finding Policy

Do not convert observations or incomplete tests into confirmed vulnerability entries. Each validated vulnerability needs evidence, affected component, impact, remediation, and a justified CVSS 3.1 score.

Prioritized Remediation Roadmap

These are proposed work targets for the capstone, not evidence that a real vulnerability has been discovered.

Priority 1 — Evidence and Safety

Target: before final submission.

- Verify repository contents and evidence filenames.
- Review Nmap target and scan results.
- Inspect Wireshark filter and representative packets.
- Ensure public repositories contain no private keys, credentials, or sensitive packet data.
- Mark untested vulnerabilities as pending.

Priority 2 — High-Priority Validated Findings

Target: within 7 days after a high-severity issue is confirmed, subject to mentor requirements and change processes.

- Validate affected component and reproduction steps.
- Apply a suitable fix or mitigation.
- Test the fix in a controlled environment.
- Record changes and retest results.

No high-severity vulnerability is asserted by this draft.

Priority 3 — Medium-Priority Findings

Target: within 30 days after confirmation, subject to risk and operational constraints.

- Review relevant code or configuration.
- Apply corrective changes.
- Add regression tests.
- Record verification evidence.

Priority 4 — Hardening Improvements

Target: within 60–90 days after approval or according to organizational policy.

- Review logging and error handling.
- Improve documentation and test coverage.
- Review key lifecycle procedures.
- Remove unnecessary exposed services where appropriate.

Timelines are proposed planning targets and must be adjusted to actual risk, business impact, and organizational policy.

Developer Patch Guidelines

SQL Injection Prevention

33. Identify database queries receiving untrusted input.
34. Replace concatenated queries with parameterized queries.
35. Apply least-privilege database permissions.
36. Test normal, invalid, and injection-like inputs.
37. Ensure database errors are not exposed to users.
38. Record code changes and test results.

XSS Prevention

39. Identify locations where user-controlled data is rendered.
40. Prefer safe text-rendering APIs.
41. Apply context-specific output encoding.
42. Remove unsafe HTML insertion where unnecessary.
43. Configure an appropriate Content Security Policy.
44. Test stored and reflected paths in the authorized lab.

Broken Access Control Prevention

45. Identify protected objects and actions.
46. Define which users/roles may access each resource.
47. Enforce authorization checks on the server.
48. Verify access using separate test identities.
49. Add regression tests for allowed and denied access.
50. Review authorization logs and error behavior.

Error Handling

Return safe client-facing errors while retaining detailed diagnostics in protected server logs. Avoid logging passwords, tokens, or sensitive data.

System Administrator and Cryptography Guidelines

Network Hardening

- Compare detected services against an approved service inventory.
- Remove unnecessary listeners and restrict exposure with firewall rules.
- Review service versions and vendor advisories after identifying actual services.
- Retest only within authorized scope.
- Document before-and-after results.

The exact services to remediate cannot be identified until the Nmap output is reviewed.

Cryptographic Hardening

- Use established cryptographic libraries.
- Generate keys with secure random generators.
- Prevent AES-GCM nonce reuse under a given key.
- Keep private/encryption keys out of source control.
- Use access controls and managed secret storage.
- Document backup, rotation, recovery, and revocation.
- Monitor dependency updates and security advisories.

Password Security

- Use bcrypt or an appropriate modern password-hashing algorithm.
- Use a unique salt for each password hash.
- Choose and periodically review a suitable work factor.
- Never store plaintext passwords.
- Protect password-reset and account-recovery flows.

Retesting, Limitations, and Conclusion

Retesting Plan

51. Record original evidence and affected component.
52. Document the approved corrective action.
53. Repeat the relevant test under the same authorized conditions.
54. Confirm whether the original behavior persists.
55. Capture new evidence.
56. Mark the result fixed, partially fixed, unresolved, or unable to retest.
57. Record residual risk.

Limitations

This report is based on project progress described during educational exercises and selected console output shared in the project record. GitHub files have not been comprehensively inspected for this draft.

- Nmap ports, services, versions, and scan parameters remain unverified.
- Wireshark capture and detailed packet contents remain unreviewed.
- Juice Shop challenge screenshots and exact technical behavior remain unverified.
- XSS and BOLA/IDOR testing remain incomplete or unverified.
- Successful bcrypt execution has not been confirmed.
- No complete CVSS-scored vulnerability set has been established.

Conclusion

The capstone demonstrates practical work across network reconnaissance, local packet analysis, web application security training, and cryptographic programming. Available records support successful AES-GCM and RSA demonstration execution and report progress in network and web application exercises. They do not yet support a complete set of validated vulnerability findings or a definitive enterprise risk rating.

The next phase is evidence validation, completion of outstanding tests, formal risk assessment, and retesting of verified findings.

Appendix — Evidence Index

Task 03 — Network Reconnaissance Audit

Repository: <https://github.com/jeevankaushik38-code/network-reconnaissance-audit>

- Nmap scan output/report: reported present; details pending review.
- Wireshark TCP-filtered screenshot: reported captured; upload and contents pending verification.
- Packet capture file: presence and publication suitability pending verification.

Task 04 — Web Application Vulnerability Assessment

Repository: <https://github.com/jeevankaushik38-code/web-application-vulnerability>

- Login Admin challenge: completion reported.
- Error Handling challenge: completion reported.
- DOM XSS alert screenshot: observed in local Juice Shop; include after scope confirmation.
- BOLA/IDOR evidence: pending.

Task 05 — Cryptography and Security

Repository: <https://github.com/jeevankaushik38-code/cryptography-security>

- AES-GCM console output: successful output shared.
- RSA signature console output: successful output shared.
- bcrypt console output: pending confirmation.
- README and repository contents: pending verification.

Final Evidence Rule

Replace pending statements only after reviewing relevant evidence. Do not claim a vulnerability, severity score, remediation, or successful retest without supporting results.

```
C:\Users\user\Task05-Cryptography>py password_hashing.py
Bcrypt password hashing demonstration
Work factor: 12
Password hashed successfully: True
Stored hash: $2b$12$LvDli00NBXM/CbzYgg6fnetA4V4/TJtGz1SD3XagYqkf045NckG1C
Correct password accepted: True
Incorrect password rejected: True
```